CLASS : B. E. (E &TC) **SUBJECT : ESD**

EXPT. NO. : 03

Aim:

Make the LED ON when the input switch interfaced with STM32F4 is pressed.

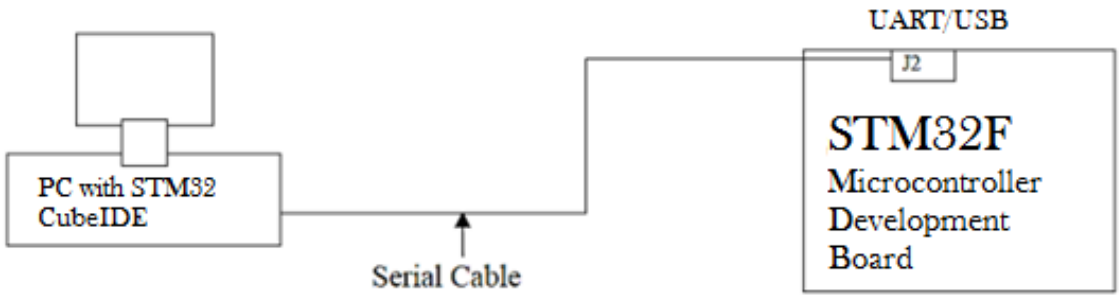
Objectives:

Create project and write code using HAL function to detect the switch status and turn ON led connected to STM32F microcontroller.

Requirements:

STM32FXX Microcontroller Development Board, Personal Computer, STM32CubeIDE Software, Connecting wires, Power Supply, etc.

Block Diagram:



Theory:

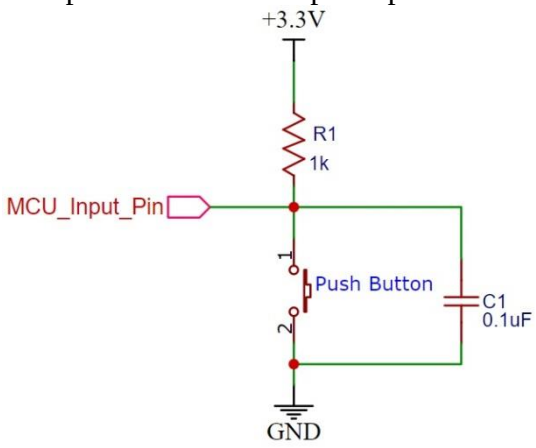
We will learn to use the GPIO pins of STM32 Nucleo board as digital input pins. In the last experiment, we have seen how to use GPIO pins as digital output pins. Now we will learn how to interface a push button with the STM32 Nuclro and read the state of a push button. Furthermore, if the push button is pressed onboard LED will turn on, and otherwise LED remains off.

Push Button Interfacing with STM32 Nucle:

When we want to interface a push button with a microcontroller, we actually want to read the state of the push button either pressed or not pressed state. When a push button is pressed or unpressed, it provides either logic high or logic low output depending on the configuration mode in which the push button circuit is designed.
There are two configuration modes to interface a push button with STM32 Nucleo board. Let's discuss both these modes one by one.

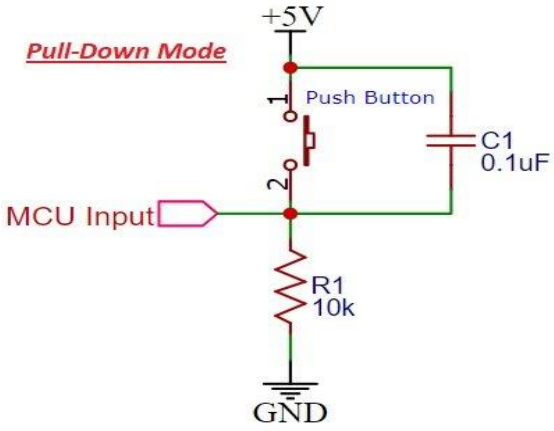
Pull-Up Mode:
In Pull-up mode, when a push button is not pressed, a logic high input appears at MCU_Input_Pin. Because a 3.3V signal appears on the input terminal through an R1 resistor. On the contrary, when the

push button is pressed, metallic contacts of the push button make contact with the ground terminal and the input terminal. Therefore, a logic low input will reflect on the digital input pin of the STM32 Nucleo board. In short, by reading this state of the push button with a digital input pin of a microcontroller, we can identify whether a push button is pressed or not. The following schematic diagram shows the connection of a push button with a pull-up resistor.



Pull-Down Mode:

In Pull-down mode, when a push button is not pressed, a logic low input appears on the STM32 Nucleo GPIO pin. Because a ground reference signal appears on the input terminal through an R1 resistor. On the contrary, when the push button is pressed, metallic contacts of the push button make contact with the +5V signal and the input terminal. Therefore, a logic high input reflects on the digital input pin of STM32 Nucleo board. The following schematic diagram shows the connection of a push button with a pull-down resistor.



Internal Pull-up and Pull-down Resistors STM32 Microcontrollers:

STM32 microcontroller's GPIO ports also have internal pull-up and pull-down resistors which can be configured through the PUPDR register. In HAL libraries, these internal pull-ups and pull-down resistors can be configured through the GPIO_InitTypeDefstruct by passing corresponding values to Pull member of the GPIO_InitTypeDef C struct. In STM32CubeIDE, we will configure it through a GUI tool.

These values which can be passed to Pull members are:

```
#define  GPIO_NOPULL        0x00000000U /*!< No Pull-up or Pull-down activation */
#define  GPIO_PULLUP        0x00000001U  /*!< Pull-up activation                 */
```

```
#define  GPIO_PULLDOWN      0x00000002U /*!< Pull-down activation        */
```

STM32 Nucleo Development Board Push Button:

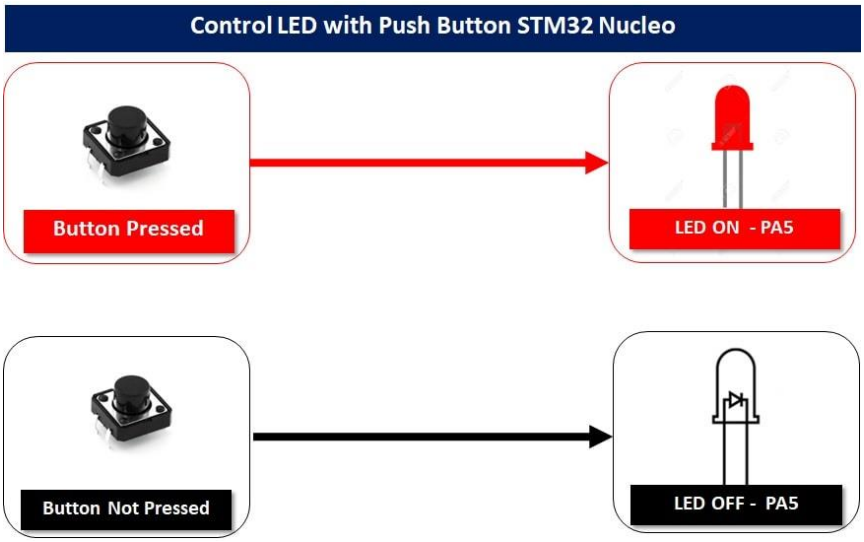
The onboard user push button is connected to PB1 digital pin through a pull-up resistor. This means when the push button is not pressed, we will get an active high signal at the PB1 pin. Similarly, when it is pressed, we will get an active low signal on the PB1 pin.

Controlling LEDs with Push Button STM32F4:

In this experiment, we will demonstrate the following example:

PB1 pin will read the output of the push button, if the state is active low, it will turn on the onboard LED of STM32 Nucleo. Because the push button is connected with the PB1 pin in pull-up configuration and under normal conditions active high output will appear on the PB1 pin.

When the push button is pressed, LED will be on, otherwise LED will remain off.



Create STM32 Nucleo Push Button Interfacing STM32CubeIDE Project:

As per guidelines provided in previous experiment, follow the steps to create project till input output configuration.

STM32 Nucleo GPIO Pins Configiuration:

Now the Device Configuration Tool window will open up. In the pinout and configuration window, we can select a specific pin and its function. A single GPIO pin may have multiple alternate functions. For example, the same GPIO pin can be used for ADC, SPI, UART peripherals, etc. But only one function can be used at a time.

1. As mentioned earlier, we will use an onboard LED of STM32 Nucleo development board. The onboard LED of Nucleo-xxxx board is connected with a PC0 pin. When you create a project for Nucleo-Fxxxx, you will see that it has already been configured for the PC0 pin. But you can change it according to your requirement.

2. Now select PB1 pin as a digital input pin by selecting option GPIO_INPUT

3. Furthermore, as you can see we have not selected a Pull-up or pull-down resistor from GPIO Pull-up/Pull-down option. Because the onboard push button of STM32 Nucelo already has an external pull-up resistor connected to it.

Now we will save our file. Press Ctrl + S. The following window will appear. Click 'Yes.' This will generate a template code for you.

Another window will appear that will ask if you want to open the perspective. Click 'Yes.'

```
if (HAL_GPIO_ReadPin(SW0_GPIO_Port, SW0_Pin) == 0)
        {
                HAL_GPIO_WritePin(RELAY_GPIO_Port, RELAY_Pin, 1);
                HAL_Delay(200);
        }
```

The HAL_GPIO_ReadPin function is available in the Hardware Abstraction Layer (HAL) library of STM32. It reads the state of a specific pin of the STM32 microcontroller, in this case, pin 1 of GPIOB (GPIO port B). It returns the state of the pin, which can be either GPIO_PIN_SET or GPIO_PIN_RESET.

```
HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_1)
```

The HAL_GPIO_WritePin function is also provided by the HAL library. It sets the state of a specific pin, in this case, pin 0 of GPIOC (GPIO port C), to the high state. It turns on the LED connected to that pin. The GPIO_PIN_SET constant is used to specify the high state.

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET)
```

The HAL library provides an abstraction layer for interacting with the hardware, making it easier to write platform-independent code. It encapsulates low-level hardware access and provides higher-level functions to interact with GPIO pins.

Building the Project:

Follow the debug and execution procedure mentioned in the previous experiment.

Conclusion: _____

_____

_____